

SMART TRANSPORTATION AND TRAFFIC ANALYTICS SYSTEM USING APACHE SPARK

Project Documentation Report

Student Name: Esha Hundekar

GitHub Repository: github.com/EshaHundekarCdac/AI_compute_Case_Study

2. Abstract

This project implements a Smart Transportation and Traffic Analytics System built on Apache Spark and PySpark, in response to Case Study 5, which asks for a metropolitan-scale traffic analytics platform. The implementation, contained in a single script, `ai_compute_case_study_.py`, ingests two traffic datasets (`1traffic.csv` and `Traffic.csv`), performs RDD-based transformations, converts the data into Spark DataFrames for filtering, grouping, joining and aggregation, and runs five Spark SQL queries to identify peak traffic hours, route-wise traffic density, average traffic volume by location, congestion hotspots, and daily/monthly trend patterns. A small ETL function consolidates both datasets into a partitioned Parquet repository, and two PySpark MLlib regression models (Random Forest and Decision Tree) are trained to predict total vehicle counts, evaluated using RMSE. The application is containerized with a Docker image based on `python:3.9-slim` with a Java Development Kit layer for Spark, and is deployed to a Kubernetes cluster (Minikube, based on the captured screenshots) using a single Deployment and a LoadBalancer Service. A GitHub Actions workflow (found in the project's Git history as `.github/workflows/deploy.yml`) automates the Docker build and push to Docker Hub on every push to the main branch. This document was produced strictly from the contents of the uploaded project archive; no functionality beyond what is present in the code, configuration files, and Git history has been assumed.

3. Introduction

3.1 Project Overview

The project is a batch analytics application for urban traffic data. It reads two independent traffic datasets, analyzes them using Spark's RDD and DataFrame APIs and Spark SQL, visualizes the results with Matplotlib/Seaborn, consolidates the data through an ETL pipeline into Parquet format, and trains regression models to predict total traffic volume. The processing script is packaged into a Docker image and deployed onto a Kubernetes cluster, with the build-and-push step automated through a GitHub Actions workflow.

3.2 Motivation

Traffic congestion in metropolitan areas is a persistent operational problem for city authorities. Aggregating sensor, junction, and traffic-situation data into a single analytics workflow allows patterns such as peak hours, high-density routes, and congestion hotspots to be identified, supporting better-informed traffic management decisions.

3.3 Problem Statement

Traffic-related data originates from multiple, differently structured sources (per-junction hourly sensor counts and 15-minute interval vehicle-category counts with a categorical traffic situation label). The project addresses the problem of ingesting, cleaning, joining, and analyzing this heterogeneous data at scale using Apache Spark, and of packaging that analysis into a deployable, repeatable containerized workflow.

3.4 Objectives

- Load and inspect two traffic datasets using Spark DataFrames and RDDs.
- Demonstrate RDD transformations, actions, key-value operations, and persistence.
- Perform DataFrame-based selection, filtering, grouping, aggregation, and joins.
- Answer traffic-analytics questions (peak hours, route density, hotspots, trends) using Spark SQL.
- Build a repeatable ETL pipeline that writes a consolidated, partitioned Parquet dataset.
- Train and evaluate regression models to predict total vehicle counts.
- Containerize the application with Docker and deploy it to Kubernetes.
- Automate the Docker build/push step with a GitHub Actions CI/CD workflow.

4. Technology Stack

The following technologies were identified directly from requirements.txt, the Dockerfile, the Kubernetes manifests, and the Git history.

Technology	Purpose	Version (as identifiable)
Python	Base runtime language for the application	3.9 (python:3.9-slim, Dockerfile)
Apache Spark / PySpark	Distributed data processing engine — RDDs, DataFrames, Spark SQL, MLlib	Not pinned in requirements.txt (unversioned pip install)
Pandas	Conversion of Spark DataFrames to Pandas for plotting (toPandas())	Not pinned in requirements.txt
Matplotlib	Chart rendering (bar, line, pie charts)	Not pinned in requirements.txt
Seaborn	Statistical chart styling (barplot, lineplot)	Not pinned in requirements.txt
PySpark MLlib	RandomForestRegressor, DecisionTreeRegressor, StringIndexer, VectorAssembler, RegressionEvaluator	Bundled with PySpark
Java (default-jdk)	Required runtime dependency for the Spark JVM	Distro default (Debian slim apt package), not pinned
Docker	Containerization of the application	Not identifiable (no docker version file)
Kubernetes	Orchestration/deployment (Deployment + Service)	apiVersion apps/v1 and v1 used in manifests
Minikube	Local Kubernetes cluster used for deployment (per OUTPUT screenshots)	Not identifiable
GitHub Actions	CI/CD automation (build and push Docker image)	actions/checkout@v4, docker/login-action@v3
Git / GitHub	Version control and remote repository hosting	N/A

5. Project Architecture

The architecture, as implemented in the project, follows a linear batch-processing pipeline that is then packaged for containerized deployment:

5.1 Data Source

Two static CSV files bundled directly into the project and the Docker image act as the data source: `1traffic.csv` (hourly per-junction vehicle counts) and `Traffic.csv` (15-minute interval vehicle-category counts with a traffic-situation label). There is no external database, streaming source, or API in this project.

5.2 Processing Pipeline (Step by Step)

- Step 1: A `SparkSession` (and its underlying `SparkContext`) is created with `appName "SmartTransportationAnalytics"`.
- Step 2: Both CSV files are read into Spark `DataFrames` with header and `inferSchema` enabled; schema and sample rows are printed.
- Step 3: Each `DataFrame` is converted to an `RDD`; `RDD`-level filter, map, and `reduceByKey` operations analyze high-traffic junctions and total vehicles per traffic situation.
- Step 4: A key-value `RDD` is built, aggregated with `reduceByKey` (a shuffle operation), persisted in memory (`StorageLevel.MEMORY_ONLY`), read twice, then unpersisted, to demonstrate `RDD` persistence.
- Step 5: `DataFrame` operations are performed: column selection and filtering, `groupBy/agg` aggregation, and an inner join between the two datasets on a derived `Date` key (day-of-month extracted from `DateTime`).
- Step 6: Both `DataFrames` are registered as Spark SQL temporary views (`traffic_lowercase`, `traffic_uppercase`) and five SQL queries are executed for peak-hour, route-density, location, hotspot, and trend analysis; each result is converted to `Pandas` and plotted with `Matplotlib/Seaborn`.
- Step 7: An ETL function (`run_traffic_etl_pipeline`) re-extracts both CSVs, derives `Date/Hour` columns, filters invalid records (`Total > 0`), joins the datasets, and writes the result as `Parquet`, partitioned by `Date`, to `centralized_traffic_repository`.
- Step 8: Feature engineering (`StringIndexer` on `Traffic Situation`, `VectorAssembler` over vehicle-count columns) prepares a feature vector; the data is split 80/20 into train/test sets.
- Step 9: A `RandomForestRegressor` and a `DecisionTreeRegressor` are trained to predict the `Total` column and evaluated with `RegressionEvaluator` (`RMSE`); the two `RMSE` values are compared to select the better-performing model.

5.3 Docker Layer

The entire script and both CSV datasets are copied into a `python:3.9-slim` image with the `JDK` installed, and the container's default command runs the script directly (`python ai_compute_case_study.py`), i.e., the analytics job executes to completion when the container starts — there is no long-running web server or API inside the container.

5.4 Kubernetes Layer

A single-replica Deployment (spark-traffic-deployment) runs the image spark-traffic-app:latest with imagePullPolicy: Never, meaning the image must already exist in the cluster's local image cache (consistent with a Minikube workflow, where the image is built directly against the Minikube Docker daemon). A LoadBalancer Service (spark-traffic-service) exposes container port 80.

5.5 CI/CD Layer

A GitHub Actions workflow, present in the repository's Git history at .github/workflows/deploy.yml (commit 61b9995, "Add CI/CD pipeline"), triggers on pushes to main, checks out the code, builds the Docker image, logs in to Docker Hub using repository secrets, and pushes the tagged image to Docker Hub. Note: this file was not present in the uploaded ZIP's working tree — it was recovered from the project's .git history.

6. Project Directory Structure

Directory tree as found in the uploaded PROJECT.zip (the .git internal folder is omitted for brevity):

```
PROJECT/
├── ai_compute_case_study_.py      # Main PySpark analytics + ML script
├── Dockerfile                    # Container build definition
├── deployment.yaml               # Kubernetes Deployment manifest
├── service.yaml                  # Kubernetes Service manifest
├── requirements.txt              # Python dependencies
├── 1traffic.csv                  # Dataset 1 (hourly junction sensor data)
├── Traffic.csv                   # Dataset 2 (15-min interval traffic situation data)
├── README.md                     # Project README
├── .gitignore                    # Ignore rules
├── OUTPUT/                       # Execution/deployment screenshots
│   ├── docker.jpeg
│   ├── docker image.jpeg
│   ├── docker final.jpeg
│   ├── kubernetes.jpeg
│   ├── kubernetes final.jpeg
│   └── kube nodes.jpeg
```

(Recovered from Git history, not present in the ZIP working tree)
.github/workflows/deploy.yml # GitHub Actions CI/CD workflow

6.1 Description of Major Files

File / Folder	Description
ai_compute_case_study_.py	Exported from a Google Colab notebook; contains all logic for Q1–Q7 (Spark init, RDDs, key-value ops, DataFrames, SQL/EDA, ETL, ML models).
Dockerfile	Builds the container image: Python 3.9-slim base, JDK install, dependency install, copies datasets and script, runs the script as the container's entrypoint command.
deployment.yaml	Defines a single-replica Kubernetes Deployment for the container image.
service.yaml	Defines a LoadBalancer Service exposing port 80 for the deployment.

File / Folder	Description
requirements.txt	Lists pyspark, pandas, matplotlib, seaborn (no version pins).
1traffic.csv / Traffic.csv	The two raw datasets consumed by the pipeline.
OUTPUT/	Screenshots evidencing Docker build/run and Kubernetes deployment on Minikube.
README.md	Existing project overview, tech stack, and structure summary.

7. Dataset Description

7.1 Dataset 1 — 1traffic.csv

Attribute	Details
Columns	DateTime, Junction, Vehicles, ID
Data types (inferred by Spark)	DateTime: timestamp/string; Junction: integer; Vehicles: integer; ID: long/string
Number of records	48,120 data rows (48,121 lines including header)
Purpose	Hourly vehicle-count readings per traffic junction, used for RDD analysis, peak-hour detection, route-density, and trend analysis.
Source	Not identifiable from the project files; no data-source documentation or link is included in the repository.

7.2 Dataset 2 — Traffic.csv

Attribute	Details
Columns	Time, Date, Day of the week, CarCount, BikeCount, BusCount, TruckCount, Total, Traffic Situation
Data types (inferred by Spark)	Time: string; Date: integer (day of month); Day of the week: string; Car/Bike/Bus/TruckCount: integer; Total: integer; Traffic Situation: string (categorical: e.g., low, normal, heavy)
Number of records	2,976 data rows (2,977 lines including header)
Purpose	15-minute interval vehicle-category counts with a labeled traffic condition, used for DataFrame aggregation, hotspot detection, and as the ML label source (Total) and feature source (Traffic Situation).
Source	Not identifiable from the project files; no data-source documentation or link is included in the repository.

Note: the join between the two datasets in both the ad-hoc DataFrame join and the ETL pipeline uses only the day-of-month (Date) as the join key, without a year or month component. Since 1traffic.csv spans November 2015 onward (based on its DateTime values) while Traffic.csv's Date column is a plain day-of-month integer, this join key is coarse and could combine unrelated days across months/years — this is called out again in Section 15 (Challenges).

8. PySpark Implementation

8.1 Data Loading

Both CSVs are loaded via `spark.read.csv(..., header=True, inferSchema=True)`, with `printSchema()` and `show(5)` used to inspect structure and sample rows for each DataFrame (`df_lowercase` for `1traffic.csv`, `df_uppercase` for `Traffic.csv`).

8.2 Data Cleaning

Cleaning is limited and targeted: within the ETL function, rows in the `Traffic.csv`-derived DataFrame with `Total <= 0` are filtered out (`df_situation_reports.filter(df_situation_reports["Total"] > 0)`) to remove invalid/empty traffic readings before the join.

8.3 Transformations

- RDD filter: keep rows where `Vehicles > 50` (high-traffic hours).
- RDD map: project rows into (Junction, Vehicles) key-value pairs.
- RDD map + `reduceByKey`: aggregate Total vehicles per Traffic Situation category.
- Column derivation: `dayofmonth(DateTime)` extracted into a new Date column on `df_lowercase` to enable joining; `hour(DateTime)` extracted in the ETL step.

8.4 Filtering

DataFrame-level filtering selects records with `CarCount > 100` from `Traffic.csv` (`df_uppercase`), representing high-volume traffic snapshots, and restricts columns to Date, Day of the week, CarCount, and Traffic Situation.

8.5 Aggregation

`groupBy("Traffic Situation").agg(...)` computes average CarCount, average BikeCount, and maximum Total per traffic-situation category. Spark SQL queries additionally aggregate by Hour, Junction, Day of the week, and Month/Day (see Section 9).

8.6 Joins

An inner join is performed twice: once ad-hoc (`joined_df`) and once inside the ETL pipeline (`centralized_analytics_df`), both joining the two datasets on the derived Date (day-of-month) column.

8.7 Feature Engineering

For the ML stage, `StringIndexer` converts the categorical Traffic Situation column into a numeric `Traffic_Situation_Index`, and `VectorAssembler` combines CarCount, BikeCount, BusCount, TruckCount, and `Traffic_Situation_Index` into a single features vector column, with Total as the regression label.

8.8 Output Generation

Two forms of output are produced: (1) the ETL pipeline writes a partitioned Parquet dataset (centralized_traffic_repository, partitioned by Date) as the consolidated analytics repository, and (2) each analysis step prints results to console and/or renders a Matplotlib/Seaborn chart (bar, grouped bar, pie, line plots) directly in the script's execution output.

9. Spark SQL

Both DataFrames are registered as temporary views — traffic_lowercase (from 1traffic.csv) and traffic_uppercase (from Traffic.csv) — and five SQL queries are executed:

9.1 Peak Traffic Hours

```
SELECT HOUR(DateTime) AS Hour, SUM(Vehicles) AS Total_Vehicles
FROM traffic_lowercase
GROUP BY Hour
ORDER BY Total_Vehicles DESC
```

Purpose: identify which hour of the day carries the highest total vehicle volume. Logic: extracts the hour component from the timestamp and sums vehicle counts per hour. Expected output: an hour-ranked table, visualized as a bar chart; the script's inline commentary notes a bimodal pattern with peaks around hour 12 and hours 19–20.

9.2 Route-Wise Traffic Density

```
SELECT Junction, AVG(Vehicles) AS Avg_Vehicles, MAX(Vehicles) AS Max_Vehicles
FROM traffic_lowercase
GROUP BY Junction
ORDER BY Avg_Vehicles DESC
```

Purpose: use Junction as a proxy for route/location to compare average and peak traffic density. Logic: groups by junction and computes mean and maximum vehicle counts. Expected output: a per-junction comparison table, visualized as a grouped bar chart.

9.3 Average Traffic Volume by Location (Day / Situation)

```
SELECT `Day of the week`, `Traffic Situation`, AVG(Total) AS Avg_Total_Vehicles
FROM traffic_uppercase
GROUP BY `Day of the week`, `Traffic Situation`
ORDER BY Avg_Total_Vehicles DESC
```

Purpose: determine average traffic volume across combinations of weekday and traffic-situation category (used by the project as a stand-in for the case study's "average travel time by location" requirement, since no travel-time or location column exists in the data). Logic: groups by day-of-week and traffic-situation, averaging Total. Expected output: a ranked table of weekday/situation combinations by average volume.

9.4 Congestion Hotspots

```
SELECT Date, `Day of the week`, Time, Total, `Traffic Situation`
FROM traffic_uppercase
```

```
WHERE `Traffic Situation` IN ('heavy', 'high')
ORDER BY Total DESC
```

Purpose: isolate the specific date/time records marked as heavy congestion. Logic: filters rows where Traffic Situation is 'heavy' or 'high', ordered by Total descending. Expected output: a ranked list of the most severe congestion events.

9.5 Daily and Monthly Traffic Trend Report

```
SELECT MONTH(DateTime) AS Month, DAY(DateTime) AS Day, SUM(Vehicles) AS Total_Vehicles
FROM traffic_lowercase
GROUP BY Month, Day
ORDER BY Month, Day
```

Purpose: build a daily/monthly trend of total vehicle volume. Logic: extracts month and day from the timestamp and sums vehicles per (month, day). Expected output: a chronological trend table, visualized as a multi-line chart (one line per month).

10. Docker

10.1 Dockerfile Contents

```
FROM python:3.9-slim
RUN apt-get update && apt-get install -y default-jdk && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY 1traffic.csv .
COPY Traffic.csv .
COPY ai_compute_case_study_.py .
CMD ["python", "ai_compute_case_study_.py"]
```

10.2 Explanation

Aspect	Detail
Base image	python:3.9-slim — a minimal Debian-based Python 3.9 image.
Installed dependencies	default-jdk (required for the Spark JVM) via apt-get; pyspark, pandas, matplotlib, seaborn via pip from requirements.txt.
Working directory	/app
Files copied in	requirements.txt, 1traffic.csv, Traffic.csv, ai_compute_case_study_.py
Build process	docker build -t spark-traffic-app . (installs the JDK, installs Python deps, copies datasets and script).
Run process	docker run spark-traffic-app — executes CMD ["python", "ai_compute_case_study_.py"], which runs the full analytics/ML pipeline to completion and exits.
Container workflow	No server process is started; the container behaves as a one-shot batch job that prints Spark output/EDA results and, per the ETL step, writes Parquet

Aspect	Detail
	output inside the container filesystem (not persisted outside the container, since no volume is mounted).

10.3 Commands Used (per README / OUTPUT screenshots)

```
docker build -t spark-traffic-app .
docker images
docker run spark-traffic-app
```

11. Kubernetes

The project defines exactly two Kubernetes manifests: a Deployment and a Service. No ConfigMap, Secret, PersistentVolume, PersistentVolumeClaim, or Ingress resource is present in the project.

11.1 deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: spark-traffic-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: spark-traffic
  template:
    metadata:
      labels:
        app: spark-traffic
    spec:
      containers:
        - name: spark-traffic-container
          image: spark-traffic-app:latest
          imagePullPolicy: Never
          ports:
            - containerPort: 80
```

Field	Value / Explanation
Replica configuration	replicas: 1 — a single pod is scheduled; no horizontal scaling or high availability is configured.
Container image	spark-traffic-app:latest, pulled from the node's local image cache only (imagePullPolicy: Never) — consistent with building the image directly inside Minikube's Docker daemon rather than pulling from a registry.
Container port	80 — exposed by the container; note the application itself is a batch script with no HTTP server, so this port mapping does not correspond to a listening service inside the container based on the code reviewed.

11.2 service.yaml

```
apiVersion: v1
kind: Service
```

```

metadata:
  name: spark-traffic-service
spec:
  selector:
    app: spark-traffic
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: LoadBalancer

```

The Service selects pods labeled app: spark-traffic and exposes TCP port 80, forwarding to target port 80 on the pod. Its type is LoadBalancer; on Minikube (as evidenced by the OUTPUT screenshots), this is typically accessed via minikube service spark-traffic-service, which opens a locally tunneled URL rather than provisioning a cloud load balancer.

11.3 Deployment Commands (per OUTPUT screenshots)

```

kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
kubectl get pods
kubectl get services
kubectl get nodes
minikube service spark-traffic-service

```

The OUTPUT/kube nodes.jpeg and OUTPUT/kubernetes final.jpeg screenshots show the Minikube node in Ready status and the spark-traffic-service running/exposed, confirming a successful local deployment.

12. CI/CD Pipeline

A GitHub Actions workflow exists in the project's Git history (commit 61b9995, "Add CI/CD pipeline") at .github/workflows/deploy.yml. This file was not included in the uploaded ZIP archive's working tree but was recovered from Git history for accuracy.

```

name: CI/CD Pipeline
on:
  push:
    branches: [ main ]
jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Build Docker Image
        run: docker build -t spark-traffic-app .
      - name: Login to Docker Hub
        uses: docker/login-action@v3
        with:
          username: ${ secrets.DOCKERHUB_USERNAME }
          password: ${ secrets.DOCKERHUB_TOKEN }
      - name: Push to Registry
        run: |
          docker tag spark-traffic-app ${ secrets.DOCKERHUB_USERNAME }/spark-traffic-
app:latest
          docker push ${ secrets.DOCKERHUB_USERNAME }/spark-traffic-app:latest

```

12.1 Stage-by-Stage Explanation

Stage	Present?	Detail
Checkout	Yes	actions/checkout@v4 pulls the repository content into the runner.
Dependency installation	No	Not a separate stage; Python dependencies are installed inside the Docker build itself, not by the workflow directly.
Build	Yes	docker build -t spark-traffic-app . builds the container image on the runner.
Docker build	Yes	Covered by the "Build Docker Image" step above.
Docker push	Yes	Logs in to Docker Hub via docker/login-action@v3 using DOCKERHUB_USERNAME / DOCKERHUB_TOKEN secrets, tags, and pushes the image.
Deployment	No	The workflow stops at the Docker Hub push; it does not run kubectl apply or otherwise deploy to a cluster.

13. Execution Flow

From start to finish, based strictly on the script's top-to-bottom execution:

- 1. SparkSession/SparkContext are created.
- 2. Both CSVs are read into DataFrames; schemas and sample rows are printed.
- 3. RDDs are derived from both DataFrames; filtering, mapping, and reduceByKey demonstrate high-traffic detection and per-situation totals.
- 4. A key-value RDD is aggregated, persisted in memory, read twice, then unpersisted.
- 5. DataFrame selection/filtering, groupBy/agg aggregation, and an inner join (on derived Date) are executed and printed.
- 6. Both DataFrames are registered as SQL temp views; five Spark SQL queries run in sequence, each followed by a Pandas conversion and a Matplotlib/Seaborn chart render.
- 7. The ETL function executes: re-extracts both CSVs, derives Date/Hour, filters invalid Total values, joins the datasets, and writes partitioned Parquet output.
- 8. Feature engineering (StringIndexer, VectorAssembler) prepares the ML feature vector; an 80/20 train/test split is created.
- 9. A RandomForestRegressor is trained and evaluated (RMSE printed); a DecisionTreeRegressor is trained and evaluated (RMSE printed); the two RMSE scores are compared and the better model is printed.
- 10. The script terminates after printing the final model comparison — there is no persistent server loop.

When run inside the Docker container, this same sequence executes automatically as the container's CMD; when deployed to Kubernetes, the Deployment starts one pod that runs this same script once and then exits/completes (the manifest does not define a Job or CronJob, so under a Deployment controller, the pod exiting could trigger a restart depending on the pod's restart policy, though restartPolicy is not explicitly set in the manifest reviewed).

14. Results

14.1 Analytical Outputs

Output	Meaning
Peak traffic hours bar chart	Shows total vehicles summed per hour of day; per the script's own inline commentary, the data shows a bimodal pattern with a midday peak (hour 12) and a larger evening peak (hours 19–20).
Route-wise density grouped bar chart	Compares average vs. maximum vehicle counts per junction; per the script's commentary, Junction 1 has the highest average load while Junction 3 shows the highest single spike (180 vehicles).
Average density bar chart	Junction 1 identified as the primary hotspot with markedly higher average density; Junction 4 identified as the least congested.
Traffic-situation pie chart	Shows the proportion of records falling into each traffic-situation category; "normal" and "heavy" dominate, "low" is least frequent.
Daily/monthly trend line chart	Shows total vehicles by day, one line per month; per the script's commentary, volumes trend upward from winter months toward late autumn/early winter, with a repeating weekly-cycle pattern.
centralized_traffic_repository (Parquet)	The ETL pipeline's consolidated, Date-partitioned output combining both datasets; this is the project's single-source analytics repository.
Model predictions & RMSE	<code>predictions.select("Total", "prediction")</code> shows sample predicted vs. actual Total vehicle counts; RMSE is printed for both Random Forest and Decision Tree, and the lower-RMSE model is reported as best.

14.2 Deployment Evidence (OUTPUT/ screenshots)

Screenshot	What it shows
<code>docker.jpeg / docker image.jpeg</code>	Docker build/image-listing output for spark-traffic-app.
<code>docker final.jpeg</code>	Container run output — Spark session logs, printed schemas/sample rows, RDD and DataFrame results.
<code>kubernetes.jpeg / kubernetes final.jpeg</code>	<code>kubectrl/minikube</code> output showing the Deployment and Service running.
<code>kube nodes.jpeg</code>	<code>kubectrl get nodes</code> output showing the Minikube node in Ready state, alongside pod/service status.

15. Challenges

- Coarse join key: joining on day-of-month only (without year/month) risks merging unrelated records across different months or years, since `1traffic.csv` spans multiple years while `Traffic.csv`'s Date is a bare day-of-month integer.
- No data validation/schema enforcement beyond `inferSchema` and a single `Total > 0` filter; malformed rows or type-inference errors would not be explicitly caught.

- Ephemeral storage: the ETL pipeline's Parquet output is written to the container's local filesystem, with no PersistentVolume defined, so results do not survive pod restarts or rescheduling in Kubernetes.
- No resource requests/limits, liveness/readiness probes, or restartPolicy defined in deployment.yaml, which can make cluster scheduling and failure recovery unpredictable, particularly for a memory-intensive Spark job.
- imagePullPolicy: Never ties the deployment to images already present on the node (workable on Minikube, but not portable to a multi-node or cloud cluster without also pushing to a registry the cluster can pull from).
- The CI/CD workflow builds and pushes the image but does not run any automated tests or trigger a Kubernetes deployment, so a manual kubectl apply step remains outside the automated pipeline.
- The container's CMD runs the full script (including matplotlib/seaborn plt.show() calls) in a headless container context, which typically requires a non-interactive backend; this is not configured in the code.
- Unversioned dependencies in requirements.txt can lead to non-reproducible builds if upstream package versions change.

16. Future Enhancements

- Use a proper composite join key (year + month + day, or a shared timestamp) instead of day-of-month alone.
- Add a PersistentVolumeClaim and mount it into the pod so the Parquet ETL output and generated charts persist beyond the pod's lifecycle.
- Externalize configuration (dataset paths, Spark tuning parameters) via a ConfigMap/Secret rather than hard-coded file names.
- Extend the GitHub Actions workflow with a test stage (e.g., pytest or a Spark job smoke test) and an automated kubectl apply / Helm deployment stage.
- Add resource requests/limits and liveness/readiness probes to deployment.yaml for more predictable scheduling on a shared cluster.
- Replace the unused LinearRegression import with an actual trained/evaluated model, or remove the unused import for cleanliness.
- Pin dependency versions in requirements.txt for reproducible builds.
- Convert the batch script into a scheduled Kubernetes CronJob (rather than a long-lived Deployment) since the workload is a one-shot batch job.
- Add an Ingress resource if the analytics results are ever exposed via a web UI/dashboard.
- Persist and version the trained models (e.g., via MLflow or a model registry) instead of only printing RMSE to console.

17. Conclusion

This project delivers an end-to-end demonstration of Apache Spark's core APIs (RDDs, DataFrames, Spark SQL) applied to a real-world traffic analytics scenario, extended into a simple ETL pipeline and two MLlib regression models for predicting total vehicle counts. The application is containerized with Docker and deployed to a Kubernetes cluster (Minikube), with the Docker build-and-push step automated through a GitHub Actions workflow. The core data-processing and modeling requirements of Case Study 5 (Q1–Q7) are implemented in

the single `ai_compute_case_study.py` script, and the DevOps requirements (Git/GitHub, Docker, Kubernetes, and CI/CD) are each represented by concrete, working artifacts in the repository, with the main gaps being automated testing, automated cluster deployment within the pipeline, and persistent storage for pipeline outputs.

18. Git Commands Used

The project's actual commit history is:

Commit	Message
be1c7c3	v1
ab323bf	Updated README file
eafee2e	Output folder
61b9995	Add CI/CD pipeline

Remote: https://github.com/EshaHundekarCdac/AI_compute_Case_Study.git

18.1 Standard Git Workflow (as applicable to this project)

Command	Purpose
<code>git init</code>	Initializes a new local Git repository in the project folder.
<code>git add .</code>	Stages all new/modified files (script, Dockerfile, manifests, datasets, README) for commit.
<code>git commit -m "v1"</code>	Creates the initial commit, as seen in the project history.
<code>git branch</code>	Lists local branches; the project uses a single main branch.
<code>git checkout -b <branch></code>	Creates/switches to a feature branch (not evidenced in this project's history, but standard practice).
<code>git merge <branch></code>	Merges a feature branch back into main, if a branching workflow is used.
<code>git remote add origin</code> https://github.com/EshaHundekarCdac/AI_compute_Case_Study.git	Links the local repository to the GitHub remote.
<code>git push -u origin main</code>	Pushes local commits to the main branch on GitHub.
<code>git pull origin main</code>	Fetches and merges the latest changes from the GitHub remote.

Command	Purpose
<code>git add .github/workflows/deploy.yml && git commit -m "Add CI/CD pipeline" && git push</code>	Adds the CI/CD workflow file, as recorded in commit 61b9995.

19. README.md

A polished README.md, based on and extending the project's existing README content, is provided below. It should replace or supplement the current README.md in the repository root.

```
# Smart Transportation Analytics: AI-Powered Traffic Management

Automated, containerized PySpark solution for analyzing metropolitan traffic
data – from RDD/DataFrame/Spark SQL analytics through an ETL pipeline
to ML-based traffic volume prediction, packaged for Kubernetes deployment.

## Project Overview
Processes two traffic datasets (junction sensor readings and 15-minute
interval vehicle-category counts), performs exploratory analysis and Spark
SQL queries, consolidates the data via an ETL pipeline into partitioned
Parquet, and trains regression models to predict total traffic volume.

## Features
- Spark session/RDD/DataFrame/Spark SQL analytics (peak hours, route
  density, hotspots, daily/monthly trends)
- Key-value RDD operations with in-memory persistence
- ETL pipeline producing a partitioned Parquet analytics repository
- Random Forest and Decision Tree regression models (RMSE-evaluated)
- Dockerized batch job
- Kubernetes Deployment + LoadBalancer Service (Minikube-tested)
- GitHub Actions workflow to build and push the Docker image

## Tech Stack
Python 3.9 | Apache Spark (PySpark) | Spark MLlib | Pandas | Matplotlib |
Seaborn | Docker | Kubernetes | GitHub Actions

## Project Structure
See Section 6 of the project documentation for the full annotated tree.

## Installation
git clone https://github.com/EshaHundekarCdac/AI_compute_Case_Study.git
cd AI_compute_Case_Study
pip install -r requirements.txt

## Usage
python ai_compute_case_study_.py

## Docker Commands
docker build -t spark-traffic-app .
docker run spark-traffic-app

## Kubernetes Deployment
eval $(minikube docker-env) # build against Minikube's Docker daemon
docker build -t spark-traffic-app .
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

```
kubect1 get pods,svc,nodes
minikube service spark-traffic-service

## Future Improvements
See Section 16 of the project documentation.

## License
Not specified in the project.
```

20. Submission Checklist

Component	Present?	Notes
Source Code	Yes	ai_compute_case_study_.py
Dockerfile	Yes	Present at project root
Kubernetes YAML Files	Yes (partial)	Deployment and Service only — no ConfigMap/PV/PVC/Ingress
GitHub-ready Structure	Yes	Contains a valid .git history and remote (GitHub)
CI/CD Pipeline	Yes (recovered from Git history)	.github/workflows/deploy.yml — build & push only, not present in the uploaded ZIP working tree
Dataset	Yes	1traffic.csv and Traffic.csv included directly in the repository
README	Yes	README.md present
Requirements File	Yes	requirements.txt (unversioned)
Documentation	This document	Generated from direct inspection of the project
Execution Screenshots	Yes	OUTPUT/ folder (Docker + Kubernetes evidence)
Model Evaluation Report	Yes	RMSE printed to console for two models; no separate report file

Missing / incomplete components: automated test stage in CI/CD; automated Kubernetes deployment stage in CI/CD; ConfigMap, PersistentVolume/PVC, and Ingress manifests; a standalone model-evaluation report file; version pins in requirements.txt; license file.

21. Workflows

21.1 Data & Analytics Workflow

```
1traffic.csv }
Traffic.csv  } -> Spark DataFrames -> RDDs (filter/map/reduceByKey)
               |
               | -> DataFrame select/filter/groupBy/join
               | -> Spark SQL (5 queries) -> Pandas -> Matplotlib/Seaborn charts
               | -> ETL (extract-transform-load) -> Parquet (partitioned by Date)
               | -> Feature engineering -> Train/Test split
```

```

└─> RandomForestRegressor / DecisionTreeRegressor
    └─> RMSE evaluation → Best model printed

```

21.2 Build & Deployment Workflow

```

Developer commits code
    |
    v
git push origin main
    |
    v
GitHub Actions: deploy.yml
  1. actions/checkout@v4
  2. docker build -t spark-traffic-app .
  3. docker/login-action@v3 (Docker Hub secrets)
  4. docker tag + docker push — image published to Docker Hub

  (Manual step, not in the workflow)
    |
    v
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
    |
    v
Minikube cluster: 1 pod (spark-traffic-container)
  exposed via spark-traffic-service (LoadBalancer, port 80)
    |
    v
Container CMD runs ai_compute_case_study_.py to completion
  (prints Spark/EDA/ML output; writes local Parquet output)

```

21.3 Local Development Workflow

- 1. Clone the repository and install dependencies from requirements.txt.
- 2. Run the script locally with `python ai_compute_case_study_.py` to validate analytics/ML output.
- 3. Build and test the Docker image locally (`docker build`, `docker run`).
- 4. Point the local Docker CLI at Minikube's daemon (`eval $(minikube docker-env)`) and rebuild so the image is available in-cluster (required by `imagePullPolicy: Never`).
- 5. Apply the Kubernetes manifests and verify with `kubectl get pods / services / nodes`.
- 6. Commit and push to main, which triggers the GitHub Actions build-and-push workflow.

22. Summary

The uploaded project is a self-contained PySpark case-study solution for smart transportation analytics, implemented as a single script that covers Spark initialization, RDD operations, key-value/persistence operations, DataFrame operations, Spark SQL-based EDA (five queries covering peak hours, route density, location averages, hotspots, and trends), an ETL pipeline producing partitioned Parquet output, and two regression models (Random Forest and Decision Tree) evaluated by RMSE. It is containerized with a Docker image built on `python:3.9-slim` plus the JDK, and deployed to a Kubernetes cluster (Minikube) via a Deployment and a LoadBalancer Service, with build/push automation supplied by a GitHub Actions workflow recovered from the project's Git history. Screenshots in OUTPUT/ substantiate that the Docker build/run and the Kubernetes

deployment were both executed successfully. The most significant gaps against a fully mature DevOps setup are: no automated tests or automated cluster deployment in the CI/CD pipeline, no persistent storage for pipeline outputs, no ConfigMap/Secret/Ingress resources, and unpinned Python dependencies — all addressed as concrete recommendations in Section 16.